

Big of n-grams: use hashing trick to prevent num of grams from getting too large.

①: for quality filtering, make $K=2$ classes (good/bad)

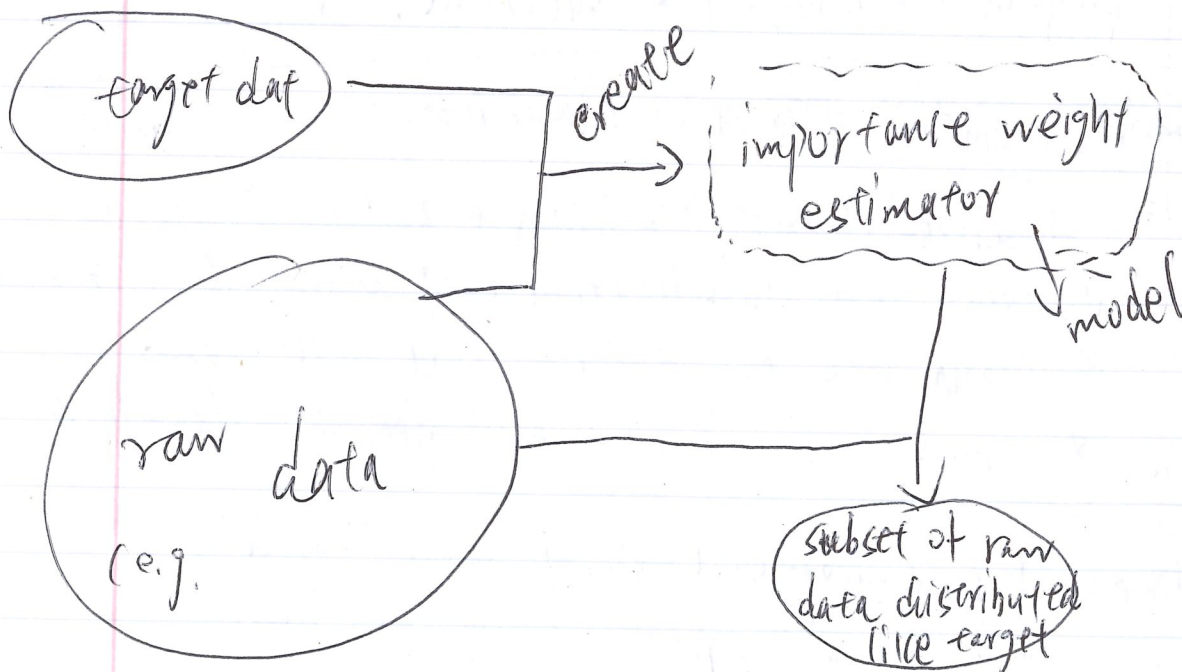
②: fastText is just a linear classifier.

③: can use large-model classifiers (BERT, Llama), but slower.

↳ dsir-main(): # Train bag of n-grams model, do importance data selection for language models via importance resampling.

①: estimate importance weights using. ran + target. data (simple bag-of-ngrams estimator)

②: select data via importance resampling ~~ETK~~



Instead of making values of subset data similar to target data, dsir makes distribution of subset data similar to distribution of target data.

def importance_sampling():

setup: { ①: Target distribution (want samples from here)
②: proposal distribution q : have samples from here

steps:

- #1: sample from q importance
- #2: compute weights over samples (w. p/q)
- #3: Resample (use hashed n-grams)

target data is too small to estimate a good model

Comparison with fastText: (DSTR is slight better than fastText)

①: Modely distribution: is a more principled approach copy diversity

②: similar computation complexity

③: Both can be improved by better text modely

Summary for Data Filtering Framework

⇒ Given target T and Raw R , find subset of R similar to T .

①: Estimate some model based on R and T and derive

②: keep examples in R based on their ^{a scoring function} score.

Instantiations of the Framework

⇒ Generative Model of T (KenLM):

1: $\text{score}(x) = p_T(x)$

2: keep examples x with $\text{score}(x) \geq \text{threshold}$ (stochastically)

⇒ Discriminative classifier (fastText):

1: $\text{score}(x) = p(T|x)$

2: keep examples x with $\text{score}(x) \geq \text{threshold}$ (stochastically)

⇒ importance resampling (DSTR)

1: $\text{score}(x) = p_T(x) / p_R(x)$; 2: Resample examples x with probability proportional to score